

Amazon ML Challenge 2026 — AI Prompt Playbook

Master Prompts for All Stages of the Competition

Use these exact prompts with your AI assistant during the hackathon for high-precision, bug-free outputs.

Stage 0: Environment & S3 Sanity Verification

Target: Jupyter Notebook / Google Colab (`notebooks/<your_name>/00_setup_test.ipynb`)

Prompt:

```
Act as a Senior Cloud ML Engineer. Write a self-contained Python script for a Google Colab / Jupyter notebook cell.
Context: We use Google Colab / local machines for compute and AWS S3 as our central cloud data store.
Requirements:
1. Check if required libraries ('pandas', 'numpy', 'rapidfuzz', 'lightgbm', 'pyarrow', 'boto3', 's3fs', 'duckdb')
2. Test read and write permissions to our central S3 bucket 's3://<INSERT_YOUR_BUCKET_NAME>/' by creating a tiny
3. Inspect available CPU count and RAM on this instance to ensure we don't encounter OOM errors later.
4. Include clear status messages (PASS/FAIL) for each check.
```

Stage 1 (Team A): Subsampled EDA & Multilingual Check

Target: Jupyter Notebook (`notebooks/<your_name>/01_eda_sample.ipynb`)

Prompt:

```
Act as an Expert Data Scientist. Write clean, modular Jupyter notebook code to perform Exploratory Data Analysis.
Requirements:
1. Read the first 50,000 rows from 's3://<BUCKET>/raw/train_source1.tsv' using pandas with sep="\t" and s3fs.
2. Check and print: distribution of 'country', percentage of Hindi (Devanagari) characters in business_name, miss
3. Provide 5 representative examples of noisy business names (abbreviations, punctuation, legal suffixes, typos).
4. Keep memory footprint under 500 MB. Provide concise explanations of what the findings mean for feature engineer
```

Stage 2 (Team A): Text Normalization, Data Filtering & Inverted Index Blocking

Target: Python Script (`src/blocking.py`)

Prompt:

Act as an Expert ML Search & Entity Resolution Engineer. We are building the blocking / candidate-generation module.

Dataset Constraints:

- Source 1 has ~2.2M records; Source 2+3 combined have ~10.3M records.
- Countries in train: US, India. Country in test additionally includes: France.
- We CANNOT do pairwise Cartesian product ($O(N*M)$) as it will crash with Out-Of-Memory.

Write a production-grade, memory-safe Python module ('src/blocking.py'):

1. 'clean_text(text)': Unicode NFKD normalization, lowercase, standardization of legal suffixes ('pvt', 'ltd', 'co').
2. 'generate_blocking_candidates(s1_df, s23_df, max_candidates=5)':
 - Partition strictly by 'country' first (never compare across different countries).
 - Use a memory-efficient inverted index based on significant word tokens (length ≥ 3) to retrieve top candidates.
 - Yield results in streaming chunks or batches to keep memory below 4GB.
3. Save filtered cleaned records to 's3://<BUCKET>/filtered/' and candidate pairs to 's3://<BUCKET>/candidates/'
4. Self-Critique: Review your code before providing it. Ensure it handles missing values (NaNs), does not crash on empty inputs.

Stage 3 (Team A): Feature Extraction & Parquet Export to Central S3

Target: Python Script (`src/features.py`)

Prompt:

Act as a Senior ML Performance Engineer. We need to compute string similarity features on candidate pairs for the training set.

Input Data:

- Candidate pairs DataFrame containing ('source1_entity_id', 'candidate_entity_id', 's1_name', 's23_name', 's1_address', 's23_address')
- Ground truth pairs to assign binary 'label' (1 for true match, 0 for negative).

Requirements:

1. Use 'rapidfuzz' (fast C++ implementation) to compute:
 - name_ratio, name_partial_ratio, name_token_sort_ratio, name_token_set_ratio.
 - addr_ratio, addr_partial_ratio, addr_token_set_ratio.
 - name_len_diff, addr_len_diff.
2. Perform GroupShuffleSplit(test_size=0.2, random_state=42) grouped strictly by 'source1_entity_id' to prevent data leakage.
3. Memory Optimization: Process candidate pairs in chunks of 50,000 rows. Use downcasted dtypes (float32, int8) to save memory.
4. Export 'train_candidates.parquet' and 'val_candidates.parquet' directly to 's3://<BUCKET_NAME>/features/full/'
5. Double-check: Confirm that no Source 1 entity appears in both the train and validation splits.

Stage 4 (Team B): Day-1 Fast Baseline Submission Pipeline

Target: Jupyter Notebook (`notebooks/<your_name>/02_quick_baseline.ipynb`)

Prompt:

Act as a Competitive Data Scientist. We need a rapid baseline script for Day 1 of the Amazon ML Challenge that gets the following done:

Challenge Rules:

- Must generate 'output/matching_results.tsv' and 'output/candidate_pairs.tsv'.
- Every test Source 1 entity must appear (1,732,544 rows).
- Singletons must have an empty 'matched_entity_ids' string.
- Final matches must be a subset of candidate pairs.
- Must pass 'python3 utils/validate_submission.py'.

Write a clean, complete script that:

1. Reads 'test_source1.tsv', 'test_source2.tsv', and 'test_source3.tsv' (from central S3 's3://<BUCKET>/raw/' or 's3://<BUCKET>/train/').
2. Uses an exact normalized name match heuristic to find high-confidence matches.
3. Formats both TSV files with correct tab delimiters and exact column headers.
4. Executes 'utils/validate_submission.py' at the end and prints the validation result.
5. Add safeguards so it doesn't crash on the French records in the test set.

Stage 5 (Team B): LightGBM Training & Macro F_{0.5} Evaluation

Target: Jupyter Notebook / Python Script ([notebooks/<your_name>/03_train_lightgbm.ipynb](#))

Prompt:

Act as an Expert Competitive ML Specialist. We need to train a LightGBM classification model on the candidate pairs to maximize the Macro F_{0.5} score. Here are some Competition Nuances:

- The evaluation metric is Macro F_{0.5} score across all Source 1 entities.
- F_{0.5} formula: $(1.25 * \text{Precision} * \text{Recall}) / (0.25 * \text{Precision} + \text{Recall})$. Precision has 2x weight over recall!
- Singletons (no true matches) get a score of 1.0 if predicted empty, and 0.0 if any false match is predicted.
- Class imbalance: true matches (label=1) are only ~5-10% of candidate pairs.

Write an end-to-end training and evaluation script:

1. Load 'train_candidates.parquet' and 'val_candidates.parquet' directly from 's3://<BUCKET>/features/full/'.
2. Set up LightGBM with parameters suited for imbalanced tabular matching ('scale_pos_weight', 'learning_rate', 'num_leaves', 'min_child_samples', 'min_child_weight', 'min_data_in_bin').
3. Implement the exact Competition Macro F_{0.5} metric function that computes entity-level scores and averages them.
4. Train the model with early stopping on validation binary logloss.
5. Save model weights to 's3://<BUCKET>/models/lgbm_model_latest.pkl'.
6. Output feature importance to show which similarities matter most.

Stage 6 (Team B): Precision-Heavy Threshold Tuning & Predictions

Target: Jupyter Notebook ([notebooks/<your_name>/04_tune_and_predict.ipynb](#))

Prompt:

Act as a Top Kaggle Grandmaster. We have trained our LightGBM model. Now we must optimize the decision threshold

Requirements:

1. Sweep decision thresholds from 0.40 to 0.95 in increments of 0.02 on the validation set.
2. For each threshold, compute the exact Macro F_{0.5} score (including singleton handling).
3. Display a table of (Threshold vs Precision vs Recall vs Macro F_{0.5}) and identify the peak threshold.
4. Apply this optimal threshold to 'test_candidates.parquet' loaded from S3.
5. Generate:
 - 'output/matching_results.tsv' (only candidates with probability $\geq$ optimal threshold).
 - 'output/candidate_pairs.tsv' (all blocking candidates evaluated by the model).
6. Ensure every single entity from 'test_source1.tsv' has a row, even if no candidates passed the threshold.
7. Run validation check using 'utils/validate_submission.py'.

Stage 7 (Team B): Submission Zip Packaging & Methodology Doc

Target: Terminal / Bash Script

Prompt:

Act as an MLOps Engineer. Write a Python / Bash script to assemble the final submission ZIP package for the Amazon

Structure required by competition:

```
<team_name>_submission.zip
├─ output/
│   ├── matching_results.tsv
│   └─ candidate_pairs.tsv
├─ src/
│   ├── features.py
│   ├── blocking.py
│   └─ ...
├─ Documentation_template.md
└─ requirements.txt
```

Requirements:

1. Verify that 'output/matching_results.tsv' and 'output/candidate_pairs.tsv' exist and are non-empty.
2. Run 'python3 utils/validate_submission.py --matching output/matching_results.tsv --candidate output/candidate_
3. Check that 'requirements.txt' pins exact versions.
4. Create the zip archive without extraneous '__pycache__' or hidden files.
5. Upload a backup copy of the zip archive to 's3://<BUCKET>/submissions/'.

Stage 8: Emergency Error / Bug-Fixing Prompt

Target: Anytime code crashes

Prompt:

I ran into an error during [INSERT STAGE NAME].

Here is the exact error traceback:

[PASTE ERROR TRACEBACK HERE]

Here is the code snippet that caused it:

[PASTE CODE SNIPPET HERE]

Context:

- Environment: Google Colab / Local Python 3.10+
- Storage: AWS S3 Central Cloud Data Store via s3fs/boto3
- Current data shape / types: [INSERT SHAPE OR DATA SAMPLE]

Please:

1. Diagnose the exact root cause of this error in 1-2 plain English sentences.
2. Provide the corrected, drop-in replacement code.
3. Verify that the fix does not introduce side effects (such as memory leaks or wrong types).